C++ pour la Finance

Projet Final

Plateforme de Trading Simulée

Objectif : Concevoir et implémenter une plateforme de trading
complète en C++17, du signal au reporting post-trade.

1er avril 2026

Table des matières

1 Contexte et Objectifs **2**
1.1 Description du projet . . . 2

2 Architecture de la Plateforme **3**
2.1 Vue d'ensemble . . . 3
2.2 Flux principal . . . 3

3 Modules Obligatoires **5**
3.1 Module 1 : Market Data Feed . . . 5
3.1.1 Événements émis . . . 5
3.1.2 Interface . . . 5
3.1.3 Format CSV . . . 6
3.2 Module 2 : Order Book & Matching Engine . . . 7
3.2.1 Alimentation par le feed . . . 7
3.2.2 Ordres de la stratégie . . . 7
3.2.3 Méthodes d'accès obligatoires . . . 7
3.3 Module 3 : Strategy Engine . . . 8
3.3.1 Architecture . . . 8
3.3.2 Stratégies à implémenter (minimum 2) . . . 8
3.4 Module 4 : Portfolio & Risk Manager . . . 9
3.5 Module 5 : Reporting . . . 10

4 Exigences Techniques **11**
4.1 Exemple de structure CMake . . . 11

5 Livrables **12**
5.1 Contenu du README.md . . . 12

6 Barème (/100) **13**
6.1 Bonus (jusqu'à +10 points) . . . 13

1 Contexte et Objectifs

1.1 Description du projet

Le projet consiste à concevoir et implémenter une **plateforme de trading simulée** en C++ qui intègre :

- un **flux de données de marché** (*Market Data Feed*)
- un **carnet d'ordres** fonctionnel (*Order Book*)
- un **moteur de matching** (*Matching Engine*)
- un **moteur de stratégies** (*Strategy Engine*)
- un **module de reporting**

◇ Simulation du cycle de vie d'un trade

Le projet simule le **cycle de vie complet d'un trade** :

$$\text{Signal} \longrightarrow \text{Exécution} \longrightarrow \text{Analyse post-trade}$$

Chaque composant correspond à un module réel utilisé en trading électronique haute performance. Ce type d'architecture est au cœur des systèmes de trading algorithmique déployés par les grandes banques d'investissement et les fonds quantitatifs.

2 Architecture de la Plateforme

2.1 Vue d'ensemble

Architecture modulaire

```
+-------------------------------------+
|  MarketDataFeed                     |
|  (parse CSV)                        |
|                                     |
| émet des événements :               |
|   - OrderAdd  (side, price, qty)    |
|   - OrderRemove (order_id)          |
|   - MarketData (bid, ask, last, ts) |
|                                     |
+-------------------+-----------------+
         |
         v
+-----------------+                      +-------------------+
|    OrderBook    | MarketData           |  Strategy Engine  |
|                 | (bid, ask, last, ts) |                   |
| reçoit Add/Remove| -------------------> | reçoit MarketData |
| maintient le book| OrderAdd            | génère des Signal |
| exécute matching | <------------------- | (BUY / SELL / HOLD)|
+--------+--------+                      +-------------------+
         |
         | trades exécutés
         v
+-----------------+
| Portfolio & Risk |
|                 |
| positions, cash |
| P&L, pré-trade  |
+--------+--------+
         |
         v
+-----------------+
|    Reporting    |
|                 |
| P&L, Sharpe,    |
| nb trades       |
+-----------------+
```

2.2 Flux principal

1. Le **MarketDataFeed** lit un fichier CSV ligne par ligne et émet des événements `OrderAdd` et `OrderRemove` qui alimentent l'`OrderBook`.

2. L'**OrderBook** maintient l'état du carnet (*bids*/*asks*) en appliquant les Add/Remove et envoie un snapshot `MarketData` au `Strategy Engine`.
3. Le **Strategy Engine** reçoit les `MarketData`, applique sa logique et génère des signaux (BUY, SELL, HOLD).
4. Quand un signal BUY ou SELL est émis, un ordre (Market ou Limit) est envoyé à l'`OrderBook` pour matching.
5. Les trades exécutés sont transmis au **Portfolio & Risk Manager** qui met à jour les positions et le P&L.
6. En fin de simulation, le **Reporting** affiche les métriques de performance.

3 Modules Obligatoires

3.1 Module 1 : Market Data Feed

Le `MarketDataFeed` est le **moteur de la simulation**. Il lit un fichier CSV et émet **uniquement des événements d'ordre** vers l'`OrderBook`.

3.1.1 Événements émis

événements émis par le MarketDataFeed

- `OrderAdd` : ajoute un ordre dans le book contient `order_id`, `side` (Bid/Ask), `price`, `quantity`
- `OrderRemove` : retire un ordre du book contient `order_id`

Origine des snapshots MarketData

Les snapshots `MarketData` (`best_bid`, `best_ask`, `last_price`...) ne proviennent **pas** du CSV. C'est l'`OrderBook` qui les génère et les transmet au *Strategy Engine* après chaque mise à jour du carnet.

3.1.2 Interface

Le `MarketDataFeed` expose une méthode `next_event()` ou un mécanisme similaire qui renvoie le prochain événement à traiter. Les événements peuvent être modélisés par :

Modélisation des événements du feed

```cpp
// Option 1 : std::variant (C++17) uniquement les événements du feed
using FeedEvent = std::variant<OrderAdd, OrderRemove>;

// Option 2 : hiérarchie de classes avec polymorphisme
class FeedEvent   { public: virtual ~FeedEvent() = default; };
class OrderAdd    : public FeedEvent { /* ... */ };
class OrderRemove : public FeedEvent { /* ... */ };

// MarketData est généré par l'OrderBook, pas par le feed
struct MarketData {
    int64_t timestamp;
    double  best_bid;
    double  best_ask;
    double  last_price;
    int64_t volume;
};
```

3.1.3 Format CSV

Format CSV attendu

```csv
timestamp,event_type,order_id,side,price,quantity
1700000001,ADD,1001,BID,42500.00,5
1700000002,ADD,1002,ASK,42510.00,3
1700000003,ADD,1003,BID,42495.00,10
1700000004,REMOVE,1001,,,,
```

Gestion d'erreurs

La gestion d'erreurs doit être **robuste** : fichiers manquants, lignes malformées, types invalides. Toute anomalie doit lever une exception avec un message explicite.

3.2 Module 2 : Order Book & Matching Engine

L'`OrderBook` reçoit les événements du `MarketDataFeed` et les ordres de la stratégie.

3.2.1 Alimentation par le feed

- `on_add(const OrderAdd&)` : insère un ordre dans le book (*bids* triés par prix décroissant, *asks* par prix croissant)
- `on_remove(const OrderRemove&)` : retire un ordre du book par son `order_id`

3.2.2 Ordres de la stratégie

Règles du matching engine

- Support de **2 types d'ordres** : `Market` et `Limit`
- Matching **tout ou rien** : un ordre est entièrement exécuté ou rejeté, pas de remplissage partiel
- Moteur de matching **price-time priority**
- Génération d'objets `Trade` à chaque exécution

3.2.3 Méthodes d'accès obligatoires

Interface de l'OrderBook

```cpp
class OrderBook {
public:
    // Alimentation par le feed
    void on_add(const OrderAdd& event);
    void on_remove(const OrderRemove& event);

    // Matching des ordres de la stratégie
    std::optional<Trade> match(const Order& order);

    // Génération du snapshot MarketData après chaque mise à jour
    // et transmission au Strategy Engine
    MarketData snapshot() const;

    // Accesseurs
    double best_bid() const;
    double best_ask() const;
    double spread()   const;

    // Affichage des N meilleurs niveaux
    void display(int levels) const;
};
```

3.3 Module 3 : Strategy Engine

3.3.1 Architecture

- Classe abstraite `Strategy` avec **interface polymorphique**
- La stratégie reçoit des `MarketData` via `on_market_data(const MarketData&)` et retourne un signal (`BUY`, `SELL` ou `HOLD`)
- Les stratégies doivent être **interchangeables** sans modifier le reste du code

Interface abstraite Strategy

```cpp
enum class Signal { BUY, SELL, HOLD };

class Strategy {
public:
    virtual ~Strategy() = default;
    virtual Signal on_market_data(const MarketData& data) = 0;
    virtual std::string name() const = 0;
};
```

3.3.2 Stratégies à implémenter (minimum 2)

Stratégies disponibles

Choisir et implémenter **au minimum 2** des stratégies suivantes :

- **Mean Reversion** Bollinger Bands : signal de retour à la moyenne
- **Trend Following** Moving Average Crossover / Momentum : suivi de tendance
- **Market Making** Spread capture avec gestion d'inventaire
- **Statistical Arbitrage** Pair trading simplifié

3.4 Module 4 : Portfolio & Risk Manager

- Suivi des **positions ouvertes**, du **cash disponible** et du **P&L réalisé**
- Mise à jour à chaque `Trade` reçu du matching engine
- **Contrôle de risque pré-trade** : position maximale par instrument

Contrôle pré-trade

Si le contrôle de risque est violé (dépassement de la position maximale), l'ordre est **rejeté** avec un log explicite. Ce mécanisme est obligatoire.

3.5 Module 5 : Reporting

Métriques obligatoires en fin de simulation

Métrique	Description
P&L total	Somme des profits et pertes réalisés sur toute la simulation
Nombre de trades	Nombre total d'ordres exécutés
Sharpe Ratio (annualisé)	$S = \frac{\bar{r} - r_f}{\sigma_r} \times \sqrt{252}$

Affichage console

Le reporting doit être **formaté lisiblement** en console. Pensez à utiliser `std::setw`, `std::fixed`, `std::setprecision` pour aligner les colonnes.

4 Exigences Techniques

Contraintes techniques obligatoires

- **C++17 minimum** utilisation des fonctionnalités modernes (`std::variant`, `std::optional`, structured bindings...)
- **Séparation stricte** headers (`.h`) / implémentation (`.cpp`)
- **Build via CMake** : le projet doit compiler avec `cmake .. && make`
- **Aucune fuite mémoire** : smart pointers obligatoires (`unique_ptr`, `shared_ptr`), pas de `new`/`delete` bruts
- **Au moins un template** utilisé de manière pertinente (ex : `TimeSeries<T>`, `EventDispatcher<T>`)
- **Algorithmes STL et lambdas** utilisés de manière significative
- **Gestion d'erreurs par exceptions** avec messages clairs

4.1 Exemple de structure CMake

CMakeLists.txt (exemple)

```
cmake_minimum_required(VERSION 3.15)
project(TradingPlatform CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(trading_sim
    src/main.cpp
    src/MarketDataFeed.cpp
    src/OrderBook.cpp
    src/MatchingEngine.cpp
    src/StrategyEngine.cpp
    src/Portfolio.cpp
    src/Reporting.cpp
)

target_include_directories(trading_sim PRIVATE include)
```

5 Livrables

Ce qui doit être rendu

1. **Code source complet** sur un dépôt Git (GitHub, GitLab...)
2. **Fichier** `CMakeLists.txt` fonctionnel
3. **Fichier(s) CSV** de données de test
4. **Fichier** `README.md` complet (voir détail ci-dessous)

5.1 Contenu du README.md

Sections obligatoires du README

1. **Présentation du projet** : description en quelques lignes du fonctionnement général
2. **Diagramme d'architecture** : schéma (texte ASCII, Mermaid ou image) montrant les modules, les événements échangés et le flux de données
3. **Instructions de build** : commandes exactes pour compiler et lancer le projet
4. **Guide d'utilisation** : comment lancer une simulation, quels paramètres modifier
5. **Description des événements** : format des `OrderAdd`, `OrderRemove`, `MarketData` et comment ils circulent entre les modules
6. **Description des stratégies** : explication de chaque stratégie implémentée, logique des signaux, paramètres configurables
7. **Résultats** : un exemple de sortie avec les métriques obtenues sur les données de test fournies
8. **Comment ajouter une nouvelle stratégie** : guide pas à pas montrant qu'il suffit d'hériter de `Strategy`

6 Barème (/100)

Grille d'évaluation /100

Critère	Détail	Points
Compilation & Build	Le projet compile sans erreur avec CMake	/5
Market Data Feed	Parsing CSV, émission correcte des événements Add/Remove/MarketData, gestion d'erreurs	/10
Order Book & Matching	Maintien du book via Add/Remove, ordres Market/Limit, matching correct, génération de trades	/20
Strategy Engine	Abstraction polymorphique propre, 2 stratégies fonctionnelles réagissant aux MarketData	/15
Portfolio & Risk	Suivi P&L réalisé, contrôle de position max, mise à jour sur trades	/10
Reporting	P&L total, nombre de trades, Sharpe ratio corrects et bien affichés	/10
Qualité du code	Const-correctness, RAII, smart pointers, nommage, séparation h/cpp, pas de code dupliqué	/10
C++ moderne	Templates, lambdas, algorithmes STL, features C++17	/5
README.md	Complet, clair, architecture et flux d'événements documentés, instructions reproductibles	/15
Total		**/100**

6.1 Bonus (jusqu'à +10 points)

Points bonus disponibles

Bonus	Description	Points
Tests unitaires	Tests avec Google Test	+4
Métriques étendues	Max Drawdown et win rate dans le reporting	+2
Export CSV	Export de l'equity curve (P&L cumulé jour par jour)	+2
Troisième type d'ordre	Stop, Iceberg ou Fill-or-Kill avec matching partiel	+2
Troisième stratégie	Avec comparaison des résultats	+2